



# 石河子大学信息科学与技术学院

## 〈数据挖掘〉课程实验报告

|       |                            |
|-------|----------------------------|
| 实验名称： | 实验 5 决策树、朴素贝叶斯以及随机森林分类算法实验 |
| 学生姓名： | 卢瑶                         |
| 学 号：  | 20211018341                |
| 学 院：  | 信息科学与技术学院                  |
| 专业年级： | 计科 2021 级 5 班              |
| 指导教师： | 刘雅辉                        |
| 完成日期： | 2024. 4. 19                |

## 实验 5 决策树、朴素贝叶斯以及随机森林分类算法实验

### 一、实验目的

- 1、掌握决策树、朴素贝叶斯、随机森林的原理。
- 2、掌握决策树、朴素贝叶斯、随机森林算法的调参技巧和使用方法。

### 二、实验原理

**决策树（Decision Tree, DT）**是一种树形结构，其中每个内部节点表示一个属性上的测试，每个分支代表一个测试输出，每个叶节点代表一种类别。常见的决策树算法有 ID3、C4.5 和 CART，分别使用信息增益、增益率和基尼指数作为属性选择度量。决策树分类器的构造不需要任何领域知识或参数设置，因此适合于探测式知识发现。

**朴素贝叶斯（Naïve Bayesian, NB）法**是基于贝叶斯定理与特征条件独立假设的分类方法。朴素贝叶斯的基础思想是：对于给出的待分类项，求解在此项出现的条件下各个类别出现的概率，哪个最大，就认为此待分类项属于哪个类别。

**随机森林（Random Forest, RF）**是集成学习的一个子类。随机森林中有许多棵分类树，它依靠于分类树的投票来决定最后的分类结果。随机森林对于多维特征的数据集分类有很高的效率，还可以做特征重要性的选择。

### 三、实验环境

使用 Weka 或 scikit-learn 实现决策树、朴素贝叶斯、随机森林算法。

scikit-learn 参考网址：

决策树 <https://scikit-learn.org.cn/view/89.html>

朴素贝叶斯 <https://scikit-learn.org.cn/view/88.html>

随机森林 <https://scikit-learn.org.cn/view/90.html>

### 四、实验数据

1. 谣言的均衡数据集 rumor.txt（包括 500 条谣言，500 条非谣言）
2. 谣言的非均衡数据集 rumorunbalance.txt（包括 50 条谣言，500 条非谣言）（注：可以采用相关方法把数据集变成均衡数据集再进行分类，并与 1 中的结果进行比较，此数据集可选做）

### 五、实验内容

本次实验，利用决策树、朴素贝叶斯、随机森林进行二分类，并比较各分类器的分类效果（注：需要调参的分类器一定去调参）。

### 六、实验步骤和结果

首先，python 编程构造决策树、朴素贝叶斯、随机森林二分类训练代码，如下所示：

```
1. import pandas as pd
2. import numpy as np
3. import matplotlib.pyplot as plt
4. from sklearn.ensemble import RandomForestClassifier
5. from sklearn.tree import DecisionTreeClassifier
6. from sklearn.naive_bayes import GaussianNB
7. from sklearn.model_selection import train_test_split
```

```

8. from sklearn.metrics import accuracy_score, confusion_matrix, roc_curve, au
   c
9. from sklearn.metrics import precision_score, recall_score, f1_score
10.
11. # 读取数据
12. df = pd.read_csv('/Users/luyao/Desktop/课内/DM/EX/实验五 分类实验-
   DT Bayes RF/rumor.txt', sep='\t')
13.
14. # 准备数据
15. x, y = df.iloc[:, 0:12].values, df.iloc[:, 13].values
16. x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, ra
   ndom_state=0)
17. feat_labels = df.columns[0:12]
18.
19. # 定义模型
20. models = {
21.     'Random Forest': RandomForestClassifier(n_estimators=100, random_state=
   0, n_jobs=-1),
22.     'Decision Tree': DecisionTreeClassifier(random_state=0),
23.     'Naive Bayes': GaussianNB()
24. }
25.
26. # 训练和评估模型
27. for name, model in models.items():
28.     model.fit(x_train, y_train)
29.     y_pred = model.predict(x_test)
30.     probas = model.predict_proba(x_test)
31.     fpr, tpr, thresholds = roc_curve(y_test, probas[:, 1])
32.     roc_auc = auc(fpr, tpr)
33.
34.     # 打印评价指标
35.     print(f"{name}:")
36.     print(" Accuracy: ", accuracy_score(y_test, y_pred))
37.     print(" Precision: ", precision_score(y_test, y_pred))
38.     print(" Recall: ", recall_score(y_test, y_pred))
39.     print(" F1-Score: ", f1_score(y_test, y_pred))
40.     print(" Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
41.
42.     # 绘制 ROC 曲线
43.     plt.figure()
44.     plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
45.     plt.plot([0, 1], [0, 1], 'k--')
46.     plt.xlim([0.0, 1.0])
47.     plt.ylim([0.0, 1.05])

```

```

48. plt.xlabel('False Positive Rate')
49. plt.ylabel('True Positive Rate')
50. plt.title(f'ROC Curve for {name}')
51. plt.legend(loc="lower right")
52. plt.savefig(f'/Users/luyao/Desktop/课内/DM/EX/实验五 分类实验-
    DT Bayes RF/ROC_{name}.png')
53. plt.show()

```

运行以上代码后，得到实验结果及 ROC 曲线如下所示：

### 6.1 决策树分类结果及 ROC 曲线

#### Decision Tree:

Accuracy: 0.655

Precision: 0.6790123456790124

Recall: 0.5612244897959183

F1-Score: 0.6145251396648045

Confusion Matrix:

[[76 26]

[43 55]]

图 1 决策树分类评价指标

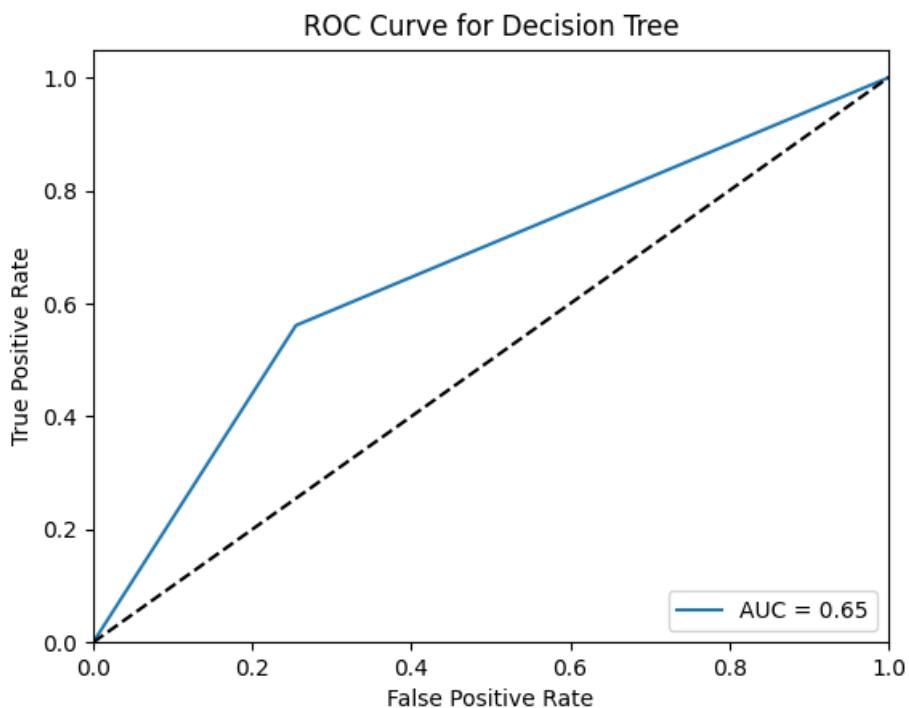


图 2 决策树 ROC 曲线

## 6.2 朴素贝叶斯分类结果及 ROC 曲线

Naive Bayes:

Accuracy: 0.585

Precision: 0.5503355704697986

Recall: 0.8367346938775511

F1-Score: 0.6639676113360324

Confusion Matrix:

```
[[35 67]
```

```
[16 82]]
```

图 3 朴素贝叶斯分类评价指标

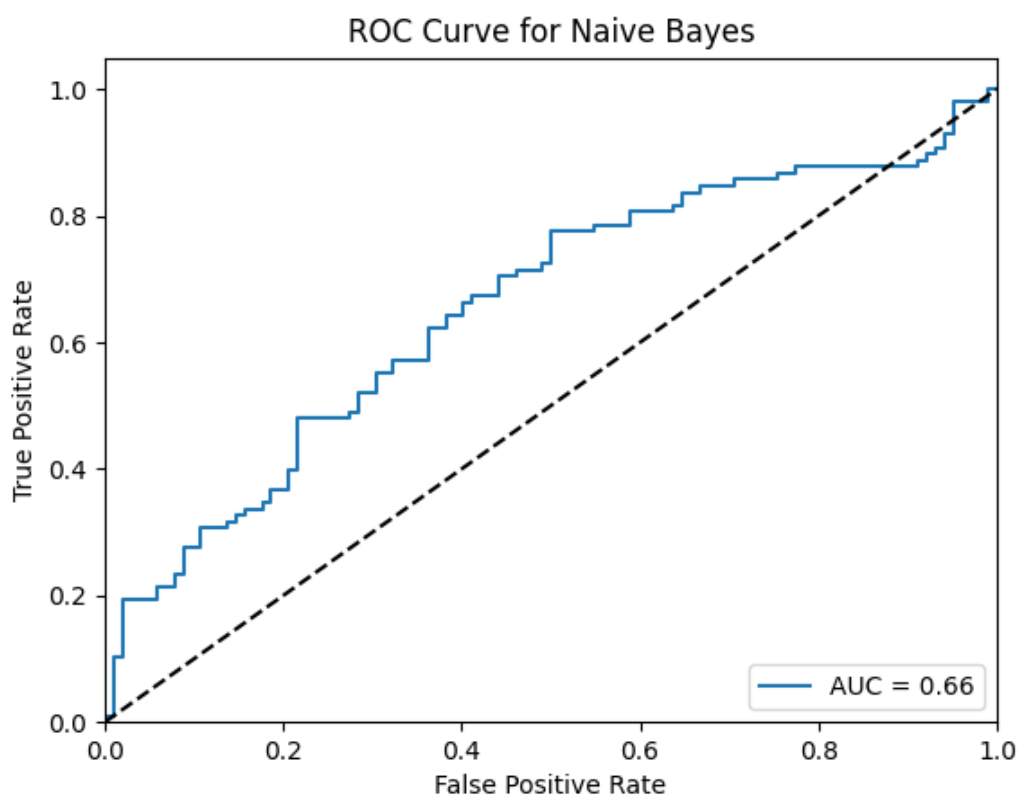


图 4 朴素贝叶斯 ROC 曲线

### 6.3 随机森林分类结果及 ROC 曲线

Random Forest:

Accuracy: 0.74

Precision: 0.7804878048780488

Recall: 0.6530612244897959

F1-Score: 0.7111111111111111

Confusion Matrix:

```
[[84 18]
```

```
[34 64]]
```

图 5 随机森林分类评价指标

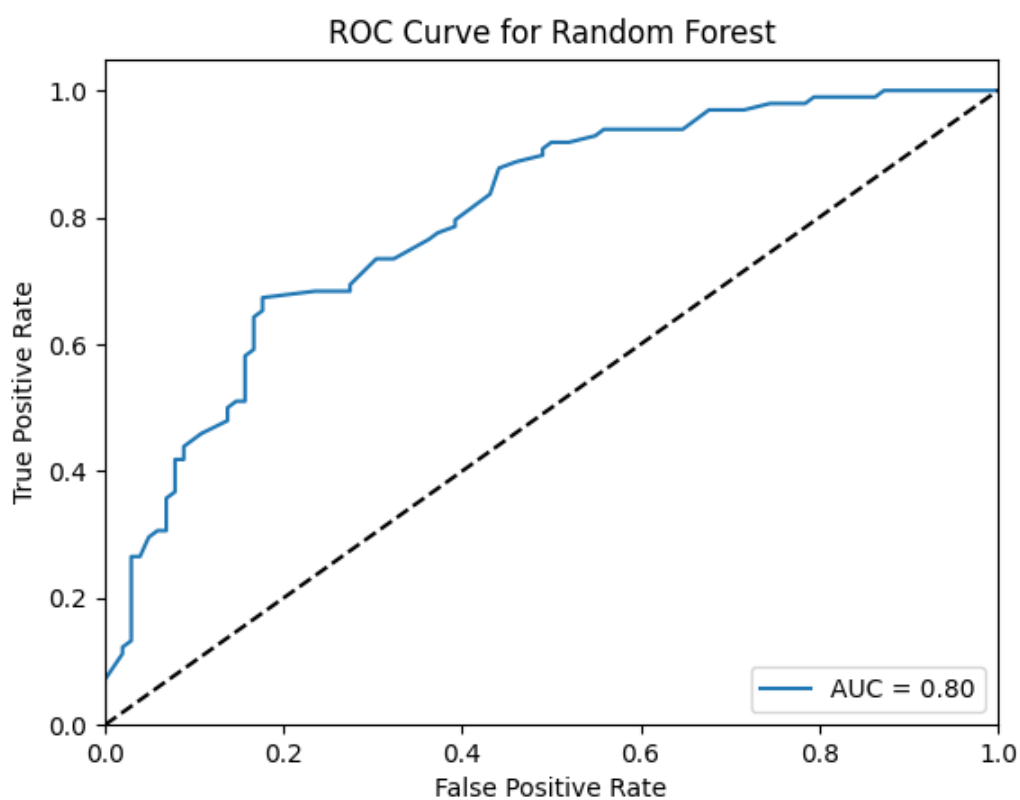


图 6 随机森林 ROC 曲线

## 七、实验结果分析

将以上 ROC 曲线汇总，制成一张总的可视化 ROC 曲线图如下所示：

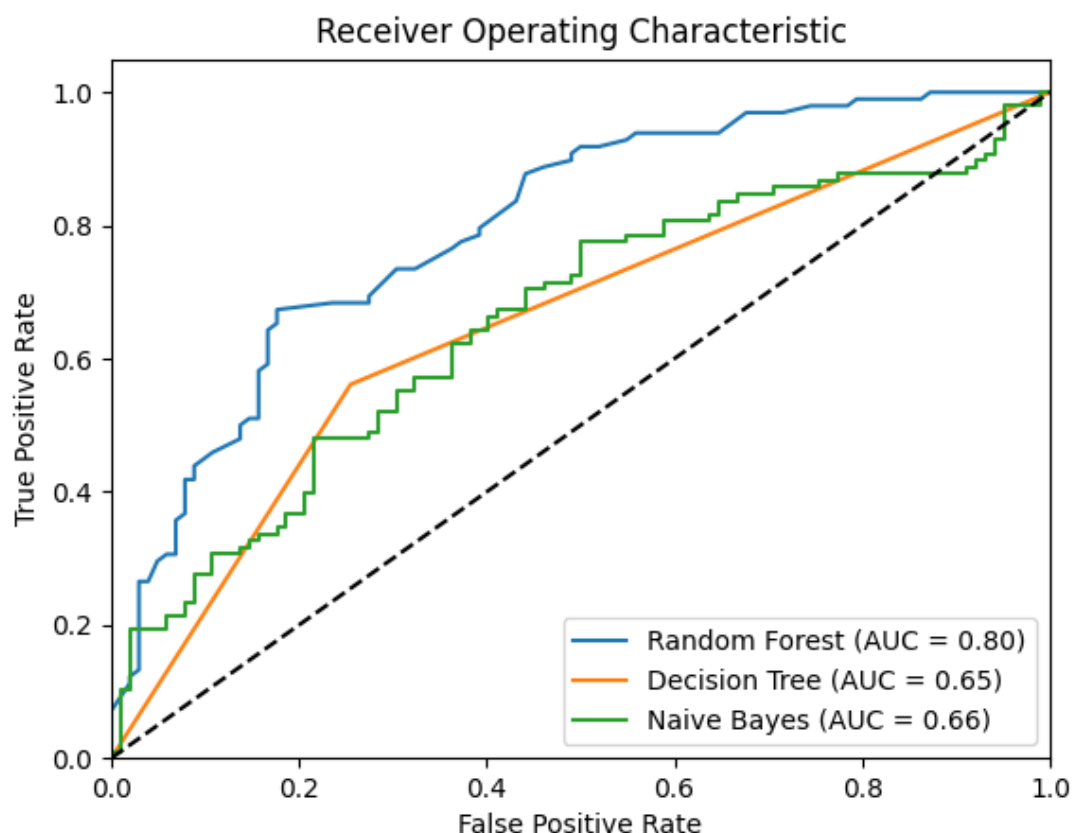


图 7 随机森林、决策树、朴素贝叶斯 ROC 曲线汇总图

### 7.1 ROC 曲线分析：

- **随机森林**：AUC 值为 0.80，这表明随机森林模型具有很好的分类能力。在 ROC 曲线上，随机森林的曲线明显高于决策树和朴素贝叶斯，这意味着在大多数阈值设置下，它正确区分正类和负类的能力更强。
- **决策树**：AUC 值为 0.65，这是三个模型中最低的，表明其分类能力一般。曲线距离对角线更近，表示其性能比随机森林和朴素贝叶斯要差。
- **朴素贝叶斯**：AUC 值为 0.66，略高于决策树，但仍低于随机森林。虽然曲线下面积只比决策树略高，但从曲线可以看出，在低假阳性率时，朴素贝叶斯的表现相对较好。

### 7.2 性能指标分析：

- **随机森林**的准确度最高 (0.74)，这表示整体上，随机森林正确分类的比例最高。其精确度也是最高的 (0.78)，意味着它在预测正类时的可靠性更好。然而，召回率较低 (0.65)，说明它未能检测出所有的真正正类实例。F1 得分 (0.71) 是平衡精确度和召回率后的结果，也是三个模型中最高的，显示了较好的平衡性能。
- **决策树**的准确度 (0.655)、精确度 (0.679)、召回率 (0.561) 和 F1 得分 (0.615) 均低于随机森林，这表明在所有指标上决策树的性能都不如随机森林。特别是在召回率上，决策树的性能显著下降，表明它错过了许多真正的正类实例。
- **朴素贝叶斯**的准确度最低 (0.585)，但其召回率最高 (0.837)，这意味着它在检测正类方面做得非常好，但同时它的精确度较低 (0.55)，这表明它产生了较多的假阳性结果。因此，尽

管朴素贝叶斯能够找到多数正类实例,但它也误将很多负类实例划分为正类。F1 得分(0.664)考虑到了高召回率的影响,但由于低精确度的拉低,这个分数并未领先于随机森林。

### 7.3 综合分析:

在选择模型时,最好的选择取决于具体的应用场景。如果目标是确保几乎不遗漏任何正类实例(例如在疾病筛查中),那么朴素贝叶斯可能是一个好选择,因为它的召回率很高。然而,这会以接受较多假阳性为代价,可能导致额外的成本或需要更多的筛查步骤。

如果需要在一个在准确度和召回率之间更加平衡的模型,随机森林是最好的选择,因为它在所有性能指标上都相对较高。

对于决策树,尽管它在某些情况下可以提供快速和易于解释的模型,但在本实验中,它在所有指标上的表现都不如其他两个模型。

总体来说,随机森林在这一系列实验中显示出最好的性能。但是,这些结论仅基于当前数据集和实验设置;不同的数据和改变模型参数会产生不同的结果。

## 八、 本次实验的主要收获和体会,使用决策树、朴素贝叶斯、随机森林分类的好处及其应用前景。

### 8.1 主要收获:

1. 理解模型特点:通过亲手实践,加深了对三种不同机器学习模型特性的理解,包括它们的优缺点和工作原理。
2. 实践模型评估:学会了如何使用准确率、召回率、精确率、F1 分数和 AUC 这些评价指标来衡量模型性能,并了解了这些指标如何反映模型在实际应用中的表现。
3. 数据处理经验:掌握了进行数据预处理、分割以及如何准备数据用于训练和测试的实用技能。
4. 模型选择与应用:通过比较不同模型在同一数据集上的性能,获得了如何根据特定的业务需求选择合适模型的经验。

### 8.2 体会:

1. 模型适用性:每种模型都有其独特适用场景。例如,朴素贝叶斯在具有大量特征的文本分类中表现良好,决策树在需要解释模型决策过程的场景中很有用,而随机森林在需要高性能且数据集复杂度较高时常常是更优的选择。
2. 模型复杂度与性能:随机森林作为集成学习方法,通常能提供比单一决策树更好的性能,但也需要更多的计算资源。朴素贝叶斯的简单性虽然有助于快速构建模型,但可能导致性能不如更复杂的模型。
3. 评价指标的重要性:不同的评价指标对于模型性能的解释是不同的。例如,召回率高的模型可能会产生很多假阳性,而精确率高的模型可能会错过一些真阳性。

### 8.3 分类的好处及其应用前景:

- 快速迭代与原型设计:三种算法都可以快速实现,便于在项目初期快速搭建原型并进行迭代优化。
- 应对不同数据类型:这些算法能够处理多种类型的数据,包括数值型和类别型数据,适用于各种不同的数据集。
- 广泛的应用领域:这些分类算法在金融风险评估、医疗疾病诊断、社交网络中的信息过滤、电子邮件垃圾分类等领域有广泛的应用。
- 提升模型性能:随着集成学习和多模型融合策略的流行,单一模型的局限性可以通过模型组合来克服,进一步提升分类效果。



随着机器学习和数据科学领域的快速发展，决策树、朴素贝叶斯和随机森林等传统算法仍然具有其坚实的理论基础和实际应用价值。特别是在解释性、计算效率和模型稳定性方面，这些算法仍然是许多复杂问题解决方案的重要组成部分。随着更多高级技术的出现，如深度学习和强化学习，这些传统算法也可以与新技术结合，用于特征工程或作为更复杂系统的一部分。因此，它们的应用前景依然广阔。